Лаборатория протоколов



Исследование и описание неизвестного сетевого протокола поверх TCP

Команда HTTP 418 · UMIRHack 2026 · кейс #02 · Мадригал
Python 3.12 · PySide6 · dpkt · pytest · Docker

Состав команды



HTTP 418 · UMIRHack 2026 · кейс #02 · Мадригал

- **Сабадаш Павел** — архитектура, capture, project, report, координация, backend, GUI · @Pasha1778
- **Предков Никита** — backend, правила, тесты · @petruan

github.com/Pavel1778/madrigal-protocol-lab

Задача: восстановить протокол, а не угадать его

Дано: захваты (pcap/pcapng) и журнал действий устройства. Спецификации нет.

Нужно: проверяемая интерпретация структуры сообщений.

Отличие от «просто распарсить»: парсер даёт один ответ; здесь видно, где он верен, а где перестаёт работать.

2

входа: байты + журнал

3

вопроса: что наблюдалось · где работает · почему это чтение

Выход — переносимая интерпретация с контрпримерами, привязанными к байтам.

01

Подход

Три модуля, связанные JSON-контрактами



capture

pcap/pcapng →

protocol + hypothesis

фрейминг, поля,

ui

настольное окно

02

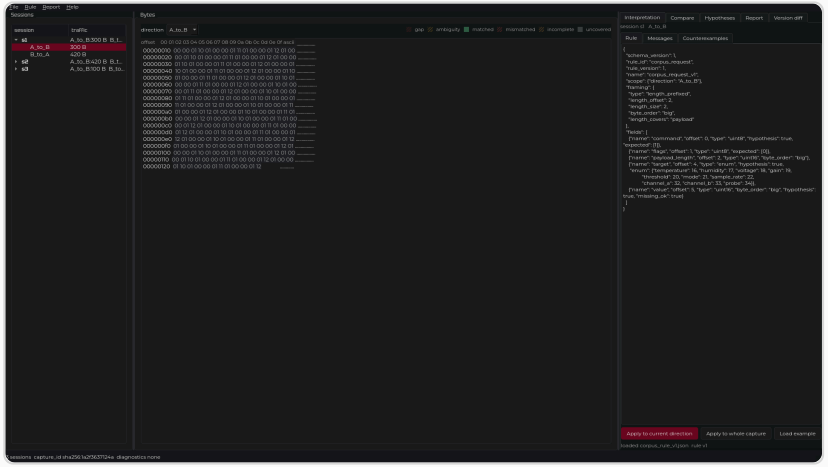
Реализация

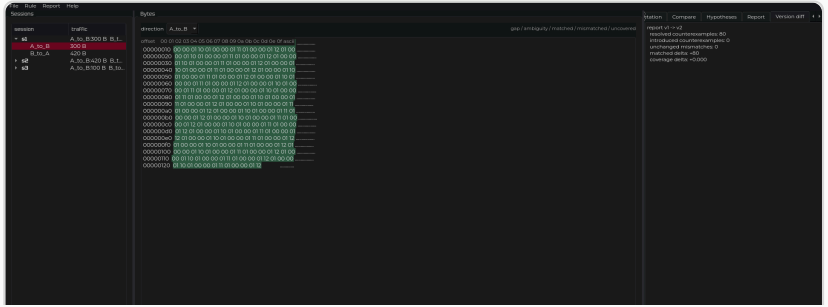
Захват потока: границы TCP ≠ границы сообщений

Сессия — экземпляр соединения, а не пара адресов: повторное использование портов даёт новую сессию.

Байты восстанавливаются по номерам последовательности; повторная передача даёт сведения о происхождении, а не лишние байты.

Пропуск не заполняется нулями — становится диагностикой, сообщение поверх него





Правила и фрейминг: одно описание на весь поток

Фрейминг: по длине, фиксированный размер, по маркерам, вручную.

Типы полей: `uint` `bytes` `enum` `string` `checksum` `computed`

Поле с пометкой `hypothesis` — предположение о смысле, а не факт.

Сообщения не перечисляются вручную: правило описывает весь поток.

```
{ "framing": {
  "type": "length_prefixed",
  "length_offset": 2 },
  "fields": [
    {"name": "command",
     "offset": 0,
     "type": "uint8",
     "expected": [1]} ] }
```

Гипотезы и контрпримеры: совпадение — не доказательство

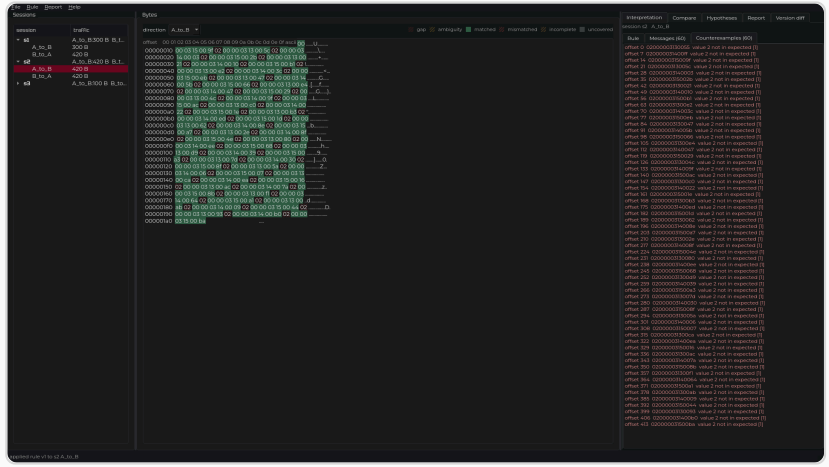
Шесть статусов по каждому сообщению:

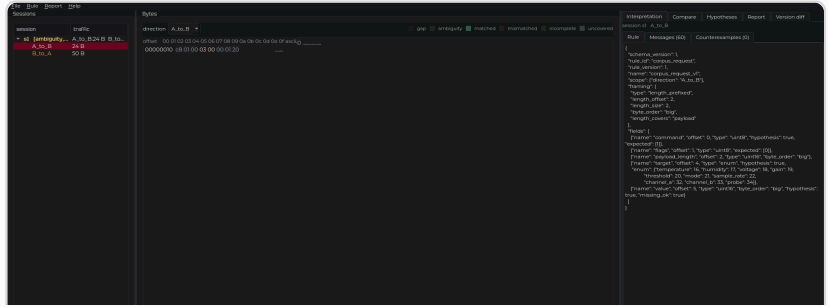
■ matched · ■ mismatched · ■ ambiguous

incomplete | uncovered | not_applicable

`mismatched` — это контрпример, привязанный к байтам и к пакету-источнику.

- Правило проверяется на **всём** корпусе.
- Контрпримеры **сохраняются**, а





Функционал: три киллер-фичи

R1 · источник

Каждое утверждение о протоколе привязано к конкретному байту, пакету и времени.

R4 · проверка

Контрпримеры сохраняются, а не отбрасываются. Совпадение на примерах ≠ доказательство.

R6 · перенос

Правило, применённое к одному захвату, применяется к новым данным без ручного описания.

Три свойства отвечают на R1, R4 и R6: происхождение, опровержимость, переносимость.

03

Демонстрация

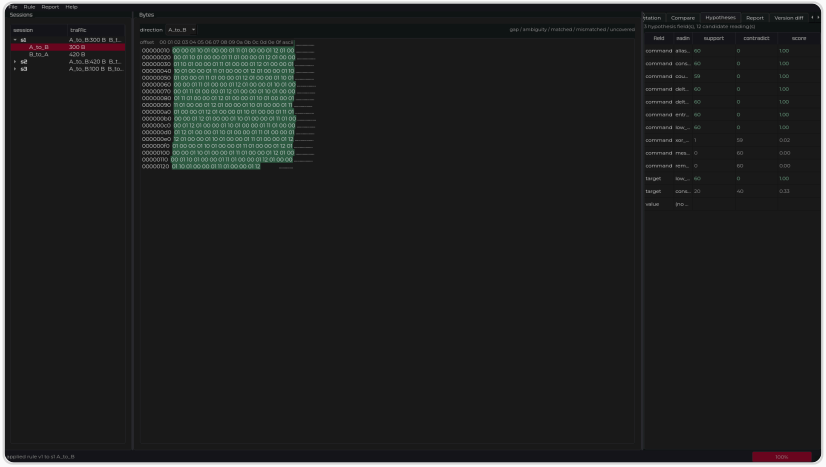
Цикл на корпусе: шесть шагов



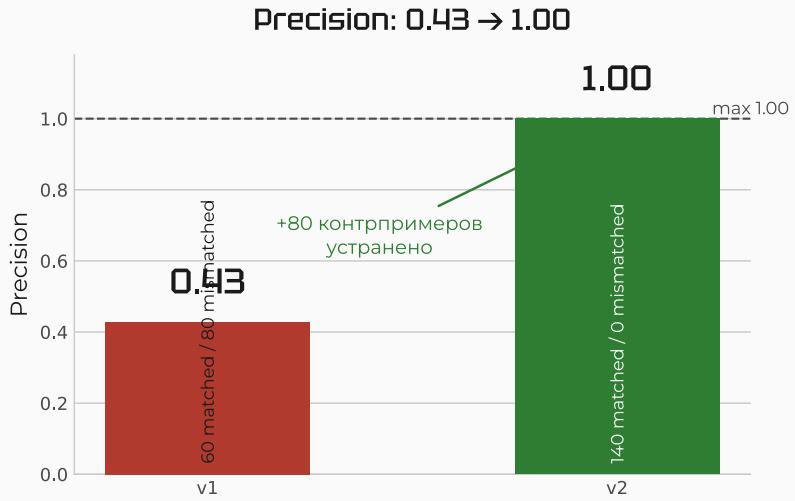
Наблюдение: байты запроса начинаются с `01 00 00 01`.

Гипотеза: по смещению 2 — длина два байта, старший вперёд.

Проверка и контрпримеры: точность 0.43 → 1.00

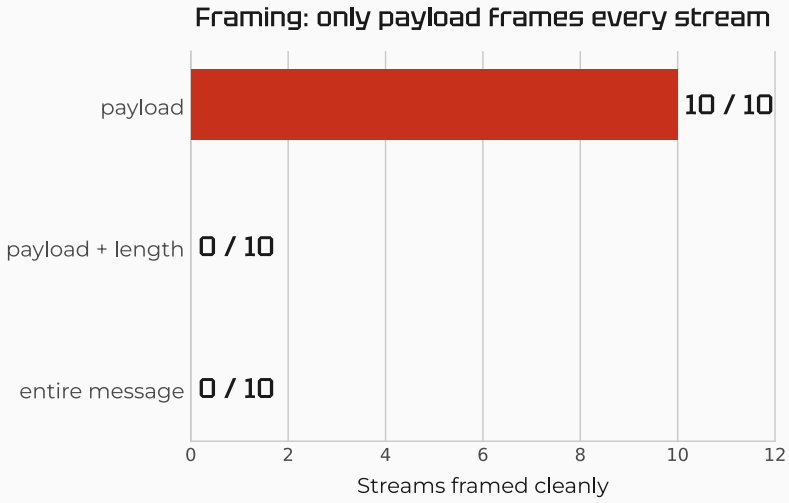


Чтение, объявленное правилом v1, даёт 0.43 (60 matched, 80 mismatched). Каждый mismatched-байт привязан к пакету-источнику и сохранён.

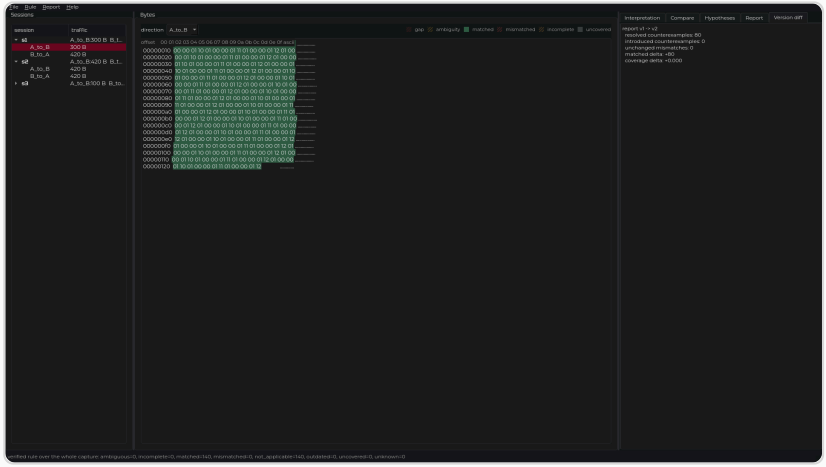


показатель	v1	v2
matched	60	140
mismatched	80	0

Сравнение версий: изменён один список, структура та же



Фрейминг `payload` разбирает 10/10 потоков; два других кандидата — 0/10.



Изменён **только** список допустимых команд: `[1]` → `[1, 2, 3]`.

Фрейминг, число полей, смещения и типы идентичны.

Контрпример как ценность: байт, который уточнил правило

Конкретный `mismatched`-байт:

поле	значение
session / direction	`s2` / `A_to_B`
message offset	`0`, length `7`
bytes	`02 00 00 03 13 00 55`
reason	`value 2 not in expected [1]`

Как уточнили правило:

```
  { "name": "command", "offset": 0,
    "type": "uint8", "hypothesis": true,
-   "expected": [1] }
+   "expected": [1, 2, 3] }
```

Один байт, привязанный к пакету-источнику, превратил 80 контрпримеров в 0 (v1 → v2).

Метрики: сводка чисел

140
matched, rule v2

0
counterexamples, v2

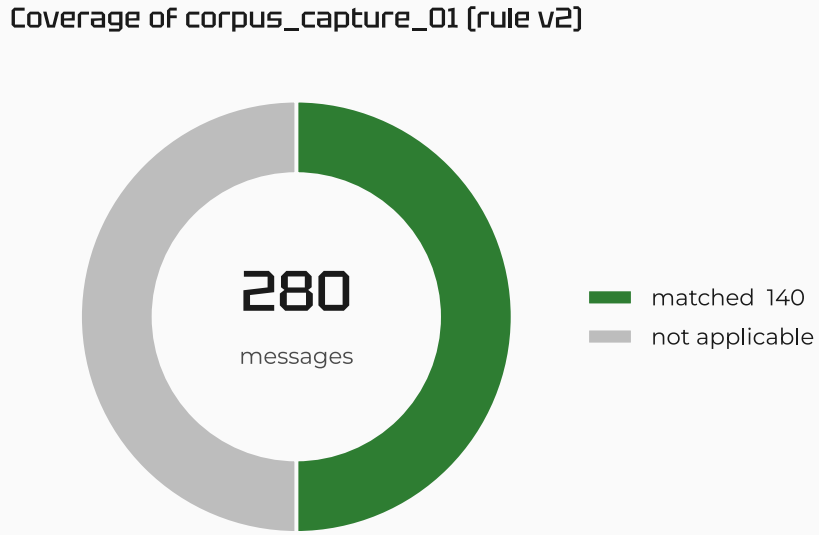
10/10
потоков разобрано

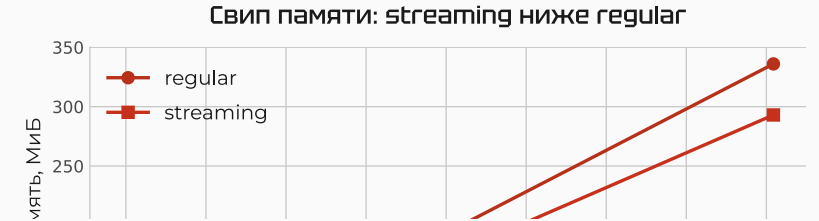
280
сообщений в захвате 01

40/40
перенос на захват 02

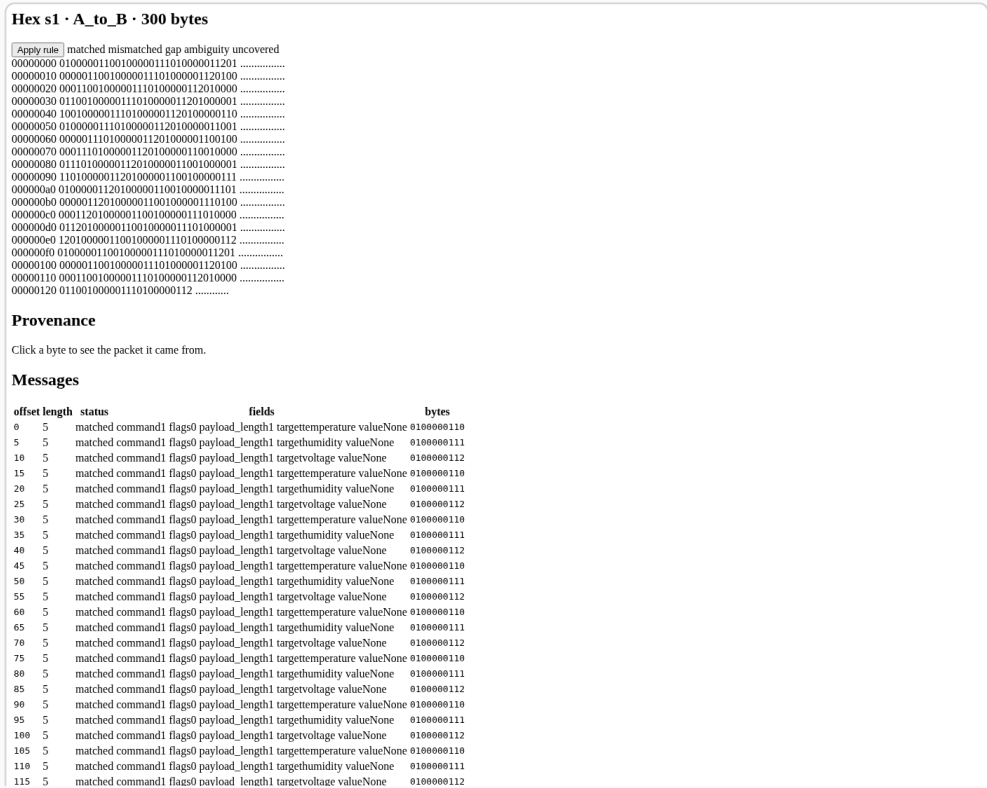
5/5
CI зелёный

Источник: `docs/METRICS.md`, `docs/AUDIT.md`.





Расширения: веб-интерфейс на localhost поверх того же ядра



Hex s1 · A_to_B · 300 bytes

Apply rule matched mismatched gap ambiguity uncovered

```
00000000 0100000110010000011101000011201 .....
00000010 0000011001000001110100000120100 .....
00000020 0001100100000111010000012010000 .....
00000030 0110010000011101000001201000001 .....
00000040 1001000001110100000120100000110 .....
00000050 0100000111010000012010000011001 .....
00000060 0000011101000001120100000110100 .....
00000070 0001101000001120100000110010000 .....
00000080 0111010000011201000001100100001 .....
00000090 1101000001120100000110010000111 .....
000000a0 0100000112010000011001000011101 .....
000000b0 0000011201000001100100000111010 .....
000000c0 0001120100000011001000001110100 .....
000000d0 0112010000011001000011101000001 .....
000000e0 1201000001100100000111010000112 .....
000000f0 0100000110010000011101000011201 .....
00000100 0000011001000001110100000120100 .....
00000110 0001100100000111010000012010000 .....
00000120 0110010000011101000001120 .....
```

Provenance

Click a byte to see the packet it came from.

Messages

offset	length	status	fields	bytes
0	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
5	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
10	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
15	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
20	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
25	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
30	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
35	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
40	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
45	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
50	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
55	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
60	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
65	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
70	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
75	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
80	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
85	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
90	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
95	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
100	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112
105	5	matched	command1 flags0 payload_length1 targettemperature valueNone	0100000110
110	5	matched	command1 flags0 payload_length1 targethumidity valueNone	0100000111
115	5	matched	command1 flags0 payload_length1 targetvoltage valueNone	0100000112

Ядро отделено от GUI контрактами JSON.

Те же `src.capture` и `src.protocol` — своей логики нет.

Ставится отдельно: `pip install -e ".[web]"`.

04

Итог

Ограничения: честный список

Смысл байта флагов не установлен: в корпусе он постоянен.

Поле значения не объяснено ни одним проверенным чтением и остаётся гипотезой.

Ответы разбираются по границам, но не по полям.

Альтернативные чтения берутся из заданного набора: смысл вне набора не будет найден.

Корпус синтетический — протокол может не совпадать с реальным устройством.

Интерпретация верна для проверенных захватов и молчит вне них.

Направления развития

IPv6 и IP-фрагментация — вне обязательного сценария; сейчас такой пакет помечается диагностикой, а не читается.

Live capture — разбор потока по мере поступления, без записи в pcap.

Полный автомат состояний — связывать запрос и ответ по транзакции, а не только по границам сообщений.

Реальные данные от производителя — корпус синтетический; на живом устройстве границы и смысл полей могут сместиться.

Ни один пункт не входит в проверенную область: это расширения, а не обещания.

Итог и ссылки

Восстановлена раскладка: код команды, флаги, длина (`uint16`, BE), полезная нагрузка.

Команда принимает `1`, `2`, `3`; смещение 4 — идентификатор параметра; 5–6 — значение при записи.

Правило переносится на второй захват и **не** переносится на посторонний.

Каждое утверждение подкреплено байтами, контрпримером или журналом.

github.com/Pavel1778/madrigal-protocol-lab

REPORT.md · docs/REFERENCE_INVESTIGATION.md · docs/demo.md

Вопросы

Команда HTTP 418 · UMIRHack 2026

741	90%	8
тест	покрытие	артефактов в релизе



Репозиторий



Релиз v1.0.0

github.com/Pavel1778/madrigal-protocol-lab